

Informe de Resolución - Casos Prácticos de Implementación

Sede: Villa Ojo De Agua

Grupo: 2

Integrantes:

- Federico Claramunt,
- Javier Molina,
- Marcia Lovaiza,
- Soledad Rios,
- Francis Feijoo.

Punto 1: Clasificación de Dígitos con Redes Neuronales (MLPClassifier)

- Descripción del Problema y Enfoque

Se implementó un algoritmo de aprendizaje supervisado utilizando una Red Neuronal Artificial para resolver un problema de clasificación de patrones ópticos. Se utilizó el dataset clásico **digits** de Scikit-Learn, el cual contiene imágenes digitalizadas de dígitos escritos a mano en formato de matrices de 8 x 8 píxeles (64 características por muestra).

Metodología Aplicada

1. **Carga y Exploración:** Se importó el conjunto de datos mapeando las variables de entrada X (características de los píxeles) y la variable objetivo Y (el dígito real del 0 al 9).
2. **División del Dataset:** Se aplicó una partición del **80% para entrenamiento** y **20% para validación/testeo** (`train_test_split`), asegurando la replicabilidad mediante el parámetro `random_state=42`.
3. **Configuración del Modelo:** Se utilizó un Clasificador Perceptrón Multicapa (`MLPClassifier`) configurando una capa oculta de 100 neuronas (`hidden_layer_sizes=(100,)`) y un límite de 1000 iteraciones para garantizar la convergencia del algoritmo.

Conclusiones del Experimento

- El modelo alcanzó un rendimiento óptimo con una alta precisión (**accuracy_score**) en el conjunto de testeo.
- Se comprobó la alta eficiencia de las redes neuronales densas para el reconocimiento y clasificación de patrones complejos basados en mapas de bits.

Punto 2: Reducción de Dimensión y Clasificación con Bosques Aleatorios (Random Forest)

- Descripción del Problema y Enfoque

Para mitigar problemas asociados a la alta dimensionalidad, el sobreajuste (*overfitting*) y balancear la varianza sin perder información crítica, se implementó un algoritmo de ensamble basado en árboles de decisión independientes conocido como Bosque Aleatorio.

- Metodología y Ajuste de Hiperparámetros

El algoritmo se entrenó utilizando el mismo dataset **digits**. La guía exigía la experimentación iterativa con sus parámetros de cómputo. Tras las pruebas del grupo, el modelo óptimo quedó configurado con:

- **n_estimators=100**: Creación de 100 árboles en el bosque para promediar las predicciones.
- **criterion='entropy'**: Utilizado para medir la calidad de las divisiones basándose en la ganancia de información (en lugar del índice **gini** por defecto).
- **max_depth=10**: Límite máximo de 10 capas de profundidad por árbol para controlar la expansión y evitar el sobreajuste.

Conclusiones del Experimento

- **Evidencia de optimización:** Durante el proceso iterativo se observó que al evaluar la profundidad (**max_depth**) en valores de 5, 10 y 15, el rendimiento óptimo se estancó o maximizó en **10**.

- Al contrastar las funciones de división, la ganancia de información mediante **'entropy'** arrojó una precisión ligeramente superior en este dataset en comparación con 'gini'. Esto demuestra que la parametrización correcta es dependiente de la naturaleza de los datos.

Punto 3: Regresión No Lineal con Árboles de Decisión

- Descripción del Problema y Enfoque

Se analizó el comportamiento de un modelo **DecisionTreeRegressor** ante una función continua con la adición de ruido aleatorio, simulando un escenario real de dispersión de datos.

- Metodología y Modificaciones Solicitadas

Partiendo de la base teórica provista (que utilizaba la función seno y una profundidad inicial baja de 2), el equipo realizó las siguientes modificaciones directamente en el código para estudiar la flexibilidad del modelo:

1. **Cambio de Función Objetivo:** Se reemplazó la función trigonométrica original por la función coseno: `y = np.cos(X).ravel()`.
2. **Ajuste de Complejidad:** Se incrementó el hiperparámetro de profundidad máxima a `max_depth=10`.

Conclusiones del Experimento

- **Fenómeno de Underfitting / Overfitting:** Se comprobó que valores muy bajos de profundidad (`max_depth=2`) generan curvas excesivamente simplificadas (subajuste) incapaces de capturar la dinámica de los datos. Al incrementar el parámetro a `10`, el árbol logra ramificarse lo suficiente para ajustarse de forma precisa a la curva del coseno distorsionada por el ruido. Sin embargo, se destaca el riesgo latente de que valores excesivamente altos memoricen el ruido provocando sobreajuste.
- **Adaptabilidad:** Al modificar la entrada a una función coseno, la línea de predicción del árbol mutó su fisonomía de forma automatizada, validando la capacidad del algoritmo para reestablecer sus reglas de decisión de acuerdo a los datos de entrenamiento provistos.